FDFED M2026 — Lab 1: Existing Application Analysis & React Planning

Project: BarelyPassing — Academic Progress & Outcome Tracking System

Team: The Boogiemen

Task 0: Application Flow

Overview

BarelyPassing is a B2B multi-tenant SaaS platform for educational institutions. It provides academic monitoring, outcome tracking, attendance management, fee compliance, and support ticketing across a 5-level role hierarchy.

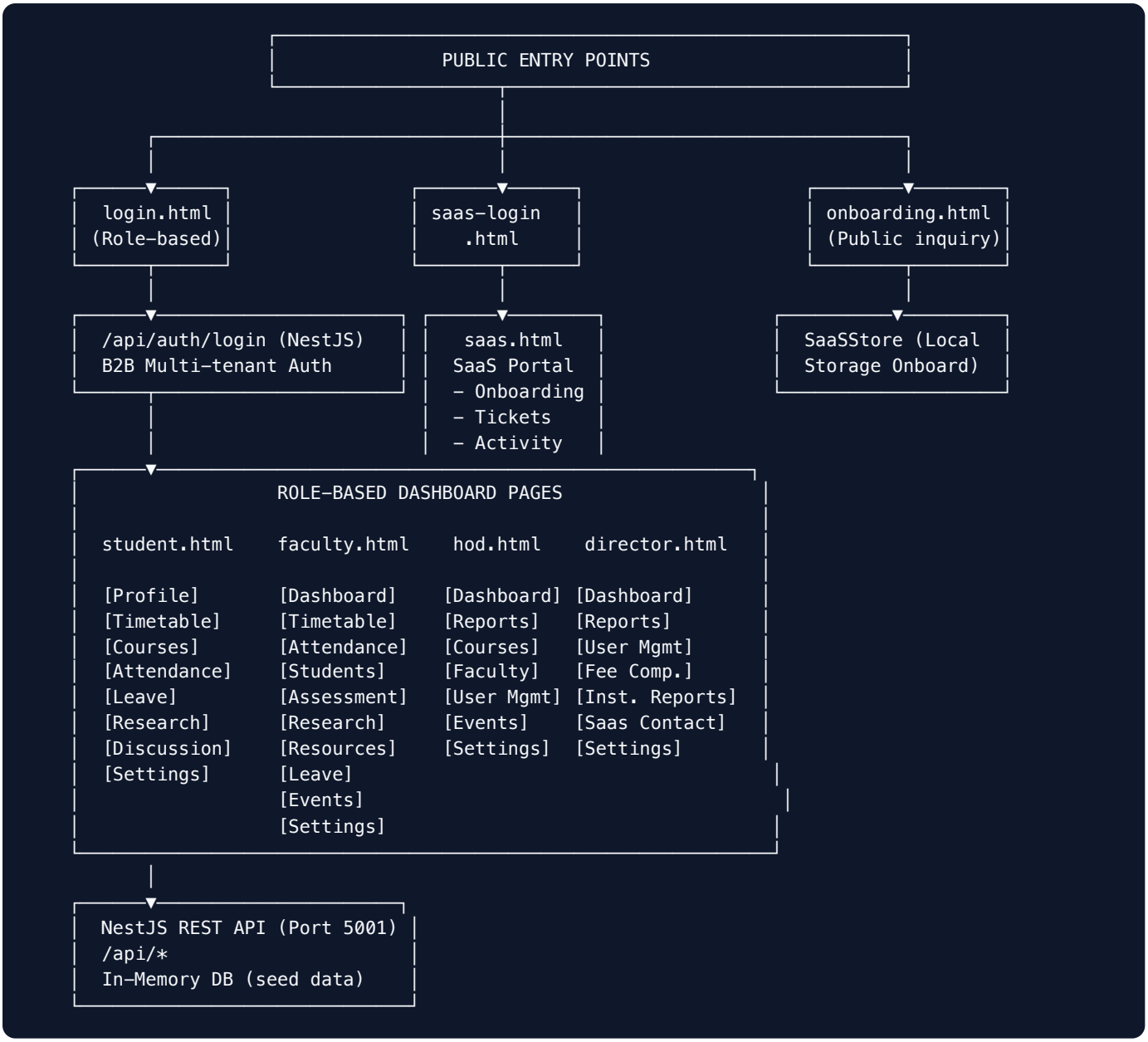
Entry Points

PORTAL	ENTRY URL	WHO USES IT
Campus User Login	`/login.html`	Students, Faculty, HODs, Directors
SaaS Support Login	`/saas-login.html`	SaaS Global Admin, Sales Lead, Tech Support
Public Onboarding	`/onboarding.html`	New institution leads / prospects

Role-Based Dashboard Routing

```
login.html
   ↓
  [Role Selected] → /api/auth/login (POST)
   ↓
  Role = student     → student.html
  Role = faculty     → faculty.html
  Role = head/HOD    → hod.html
  Role = director    → director.html
  Role = PLATFORM_*  → saas.html (via saas-login.html)
```

Application Flow Diagram



Key User Workflows

WORKFLOW	ACTORS INVOLVED	PAGES
Student views attendance	Student	`student.html` → `/api/students/me/attendance`
Faculty marks attendance	Faculty	`faculty.html` → `/api/attendance`
Student applies for leave	Student	`student.html` → `/api/leave`
HOD approves leave	HOD	`hod.html` → `/api/leave/:id`
Faculty enters marks	Faculty	`faculty.html` → `/api/marks`
Director views overview	Director	`director.html` → `/api/reports/overview`
Institution onboards	Director	`onboarding.html` → SaaSStore → `saas.html`
SaaS responds to ticket	SaaS Admin	`saas.html` → SaaSStore → `director.html`

Task 1: Repeated UI Elements

UI ELEMENT	PAGES / LOCATION	HOW IS IT REPEATED?	WHY COULD REUSE BE USEFUL?
Sidebar Navigation	`student.html`, `faculty.html`, `hod.html`, `director.html`	Same sidebar structure (brand, user info, nav items, logout button) duplicated in all 4 HTML files	One change (e.g. styling, logout logic) must be made 4 times separately
Top Header Bar	`student.html`, `faculty.html`, `hod.html`, `director.html`	`<div class="top-header-bar">` with `#page-title`, semester badge, and semester number repeated on every page	Consistent header experience; changes only need to be in one place
Stats / KPI Cards	`student.html` (CGPA, Attendance), `faculty.html` (student count), `director.html` (reports overview), `hod.html`	Cards showing a metric label and large value, styled identically with `stats-card` class	Single reusable `<StatCard label="..." value="..." />` component
Data Tables (CRUD)	User Management, Fee Compliance, Leave Management, Research Projects, Events, Resources across all 4 dashboards	Table with `<thead><tr><th>...</th></tr></thead><tbody>` built via `innerHTML` in fixes.js — same structure repeated 10+ times	A single `<DataTable columns={...} rows={...} />` component handles all tables
Modals / Dialogs	All pages: Edit User Modal, Add Course Modal, Leave Modal, Forgot Password Modal, Support Ticket Modal	Each modal has the same backdrop + window + header + body + footer structure repeated in HTML and toggled by `openModal(id)` / `closeModal(id)`	A `<Modal title="..." isOpen={...} onClose={...}>` component eliminates duplicated structure
Role Filter Pills / Tabs	User Management (director, hod), SaaS onboarding leads	Inline `<button>` pills rendered dynamically using template literals	A `<FilterTabs options={...} active={...} onChange={...} />` component
Toast / Snackbar Notifications	All pages via `showToast()` in fixes.js	`div` injected into DOM with identical CSS on every call across all pages	A `<Toast />` React component using state/context replaces the manual DOM injection
Form Input Groups	Login forms, Add User modal, Add Course modal, Leave Application form, Forgot Password modal	`<div class="input-group"><label>...</label><div class="input-icon-wrapper"><input></div><div class="error-msg"></div></div>` copied in every form	A `<FormInput label="..." error="..." type="...">` component
Loading / Empty State	All data-fetching sections (`el.innerHTML = '<p>Loading...</p>'`)	Identical inline loading placeholder and error message strings written in every render function	A `<LoadingSpinner />` and `<EmptyState message="..." />` component

UI ELEMENT	PAGES / LOCATION	HOW IS IT REPEATED?	WHY COULD REUSE BE USEFUL?
Badge / Tag Pills	Role badges in User Management, Status badges in Leave/Fee tables, Plan badges in Onboarding	`<span style="background:#e2e8f0;border-radius:4px;...">ROLE</span>` patterns repeated everywhere	A `<Badge color="..." label="..." />` component

React Connection:

Each of these repeated elements is a natural React component. Instead of 4 copies of the sidebar HTML, one `<Sidebar role={currentRole} />` component can conditionally render the correct nav items.

Task 2: Dynamic Behaviour

PAGE / FEATURE	USER ACTION	WHAT CHANGES?
`login.html`	Clicks a role card (Student / Faculty / HOD / Director)	The login form title, subtitle, and email placeholder change; the selected role card gets the `.active` CSS class
`login.html`	Submits the login form	Button text changes to "Authenticating..." and is disabled; on success, the page redirects to the role's dashboard
`login.html`	Clicks "Forgot Password"	A multi-step modal appears with 3 steps: email entry → OTP display → new password
`student.html`	Clicks any sidebar nav item	The active view section (`div.view-section`) is shown; all others are hidden; page title in the header updates; data is fetched and rendered
`student.html`	Loads Attendance view	Attendance percentage bars animate; a per-course breakdown table appears with colour-coded pass/fail indicators
`student.html`	Loads Timetable view	A weekly grid is rendered with colour-coded day columns and class type icons (Lab 🔬 / Lecture 📖 / Tutorial ✏️)
`faculty.html`	Clicks "Mark Attendance" for a course	A student list table appears; each row has Present/Absent toggle buttons
`faculty.html`	Submits attendance	Records are sent to the API; a success toast appears; the button resets
`faculty.html`	Clicks "Enter Marks"	A marks entry table appears per assessment with student rows; marks are locked once entered
`hod.html`	Opens a leave application	The application row expands or a modal appears with Approve/Reject actions
`director.html`	Clicks "User Management"	The user table loads with role filter tab pills (All / HODs / Faculty / Students); clicking a tab instantly filters the table
`director.html`	Clicks "Edit" on a user	An Edit User modal opens pre-filled with that user's data
`director.html`	Opens "Contact SaaS Support"	A support ticket modal appears; on submit, a ticket is created in SaaSStore and a toast confirms

PAGE / FEATURE	USER ACTION	WHAT CHANGES?
`saas.html`	Clicks an Onboarding Lead row	"Contact Lead" and "Invoice & Pay Link" buttons appear in the row
`saas.html`	Clicks "Invoice & Pay Link"	A B2B Invoice Modal opens; on confirm, a B2B Checkout Modal opens with total calculation (base + 18% GST)
`saas.html`	Completes payment	Lead status updates to "Paid"; activity log records the event; success toast appears
All dashboards	Logs out	All localStorage keys are cleared; page redirects to `login.html` or `saas-login.html` depending on role
All dashboards	JWT token expires or is missing	`requireAuth()` runs and shows an animated "Access Denied" screen with a 5-second countdown; all session data is cleared

React Connection:

All of these UI changes are currently driven by manual DOM manipulation. In React, each would be modelled as **state**: `activeView`, `selectedRole`, `isModalOpen`, `filterTab`, `users`, `tickets`, etc. When state changes, React re-renders the affected part of the UI automatically.

Task 3: Existing JavaScript Analysis

JAVASCRIPT	WHAT DOES IT DO?	PAGE / FEATURE	WHAT UI DOES IT AFFECT?
`document.getElementById('usersTable')`	Finds the User Management table container	Director, HOD → User Management	Entire user list is rebuilt by setting `el.innerHTML = ...`
`document.querySelectorAll('.view-section')`	Selects all view panels	All dashboards (`switchView()`)	Hides all `.view-section` panels, then shows the clicked one
`document.querySelector('.role-option[data-role="${role}"]')`	Finds the selected role card on login	`login.html` → `selectRole()`	Adds/removes `.active` CSS class to highlight selected role
`addEventListener()` (DOMContentLoaded)	Waits for DOM to fully load before running auth check and rendering	All pages	Triggers `requireAuth()`, `renderDashboard()`, `renderStudentProfile()`, etc.
`el.innerHTML = ...` (template literals)	Builds entire table/card HTML as a string and injects it	Every render function in `fixes.js`	Renders user tables, course cards, timetable grids, fee records, discussion posts

JAVASCRIPT	WHAT DOES IT DO?	PAGE / FEATURE	WHAT UI DOES IT AFFECT?
`el.innerHTML = '<p>Loading...</p>'`	Shows a loading placeholder before data arrives	All async render functions	Replaces section content temporarily while API call is in flight
`innerText` / `textContent`	Updates page title, user name, badge text	`switchView()`, sidebar user info	Updates `#page-title`, `#sidebar-user-name`, stat counters
`el.classList.add('active') / remove('active')`	Highlights the selected sidebar nav item and active view section	All dashboards, `login.html`	Sidebar item styling, view panel visibility
`style.display = 'flex' / 'none'`	Shows and hides modals	`openModal()` / `closeModal()` in `fixes.js`	All modals (Edit User, Add Course, Leave modal, Forgot Password, SaaS ticket)
`document.createElement('div')`	Creates toast notification elements dynamically	`showToast()` in `fixes.js`	Injects a success/error/warning snackbar into `document.body`, auto-removes after 3.5s
`localStorage.getItem/setItem`	Reads and writes JWT token, user object, tenant, SaaS tickets, onboarding leads, activity logs	`state.js` (Auth + SaaSStore)	Controls login state, role-based routing, inter-portal communication
`window.location.href = '...'`	Hard-navigates to a different HTML page	`state.js` login/logout, `requireAuth()`	Causes a full page reload on every route change
`document.documentElement.style.cssText = ...`	Completely replaces page body HTML for the "Access Denied" screen	`state.js` → `_showAccessDenied()`	Wipes out the entire DOM and replaces it with an error screen
`setTimeout(() => ..., 3500)`	Delays toast removal and access-denied countdown	`showToast()`, `_showAccessDenied()`	Toast disappears; redirect countdown fires

JAVASCRIPT	WHAT DOES IT DO?	PAGE / FEATURE	WHAT UI DOES IT AFFECT?
`onmouseover` / `onmouseout` inline attributes	Adds hover effects to course cards and timetable slots	`student.html` course cards, timetable grid	`boxShadow` and `transform` style changes

> **React Connection:**

> In React, none of these manual DOM manipulations would be needed. `innerHTML` and template literals are replaced by **JSX**. `classList` changes become **conditional class names**. `style.display` toggling becomes **conditional rendering** (`isModalOpen && <Modal />`). `localStorage` reads are replaced by a **global state store** (Context API or Redux). Page navigation becomes **React Router** — no full-page reloads.

Task 4: NestJS API Analysis

FEATURE	HTTP METHOD	API ENDPOINT	PURPOSE
Login	POST	`/api/auth/login`	Authenticate user with email, password, and tenant_code; returns JWT and role-based redirect path
Student Profile	GET	`/api/students/me`	Get the logged-in student's profile (name, branch, CGPA, section)
Student Attendance	GET	`/api/students/me/attendance`	Get overall and per-course attendance summary
Student Courses	GET	`/api/students/me/courses`	Get enrolled courses with marks %, attendance %, and syllabus progress
Student Timetable	GET	`/api/student-timetable`	Get weekly timetable filtered by enrolled courses only
All Courses	GET	`/api/courses`	Get all available courses
Create Course	POST	`/api/courses`	Faculty/Admin creates a new course
Timetable Grid	GET	`/api/timetable?section=A`	Get timetable grid for a specific section
Faculty Timetable	GET	`/api/timetable/faculty`	Get timetable for the logged-in faculty member
Assessments	GET	`/api/assessments?faculty_id=...`	Get all assessments, optionally filtered by faculty
Create Assessment	POST	`/api/assessments`	Faculty creates a new assessment with weightage
Get Marks	GET	`/api/marks?assessment_id=...`	Get marks entries; locked once entered
Record Marks	POST	`/api/marks`	Faculty records marks (locked, cannot overwrite)

FEATURE	HTTP METHOD	API ENDPOINT	PURPOSE
Submissions	GET	`/api/submissions`	Students see own; faculty see all
Create Submission	POST	`/api/submissions`	Student submits work for an assessment (re-submission allowed)
Attendance Today	GET	`/api/attendance/today/:courseId`	Get enrolled students for faculty to mark present/absent
Record Attendance	POST	`/api/attendance`	Bulk record attendance for a course session
Discussions	GET	`/api/discussions`	Get all discussion posts with reply counts
Discussion Detail	GET	`/api/discussions/:postId`	Get one post with all replies
Create Post	POST	`/api/discussions`	Any user creates a new discussion post
Reply to Post	POST	`/api/discussions/:postId/replies`	Any user replies to a discussion
Research Projects	GET	`/api/research`	Get projects (own for student/faculty, all for admin)
Update Research Status	PATCH	`/api/research/:id/status`	Faculty updates project status
Update Research Progress	PATCH	`/api/research/:id/progress`	Any member updates progress % or notes
Create Research	POST	`/api/research`	Faculty creates a new BTP/research project
Events	GET	`/api/events`	Get all scheduled institutional events
Create Event	POST	`/api/events`	Admin/Faculty schedules a new event
Update Event	PUT	`/api/events/:id`	Edit an event
Delete Event	DELETE	`/api/events/:id`	Delete an event
Leave Applications	GET	`/api/leave`	Students see own; admin/faculty see all
Apply Leave	POST	`/api/leave`	Student submits leave application
Approve/Reject Leave	PATCH	`/api/leave/:id`	HOD/Admin updates leave status
All Users	GET	`/api/admin/users`	Admin panel user list
Create User	POST	`/api/users`	Admin creates a new user (also creates student/faculty profile)
Update User	PUT	`/api/users/:id`	Admin edits a user's details or role
Delete User	DELETE	`/api/users/:id`	Admin removes a user
Reports Overview	GET	`/api/reports/overview`	KPI summary (total students, faculty, attendance %, fee compliance %)
At-Risk Students	GET	`/api/reports/at-risk`	List of students with CGPA < 6.5 and low attendance
Resources	GET	`/api/resources`	Get all physical/lab resources

FEATURE	HTTP METHOD	API ENDPOINT	PURPOSE
Create Resource	POST	`/api/resources`	Admin adds a new resource
Update Resource	PUT	`/api/resources/:id`	Update resource availability/status
Fee Records	GET	`/api/fees`	Get all fee records with paid/pending/overdue summary
Mark Fee Paid	PATCH	`/api/fees/:id/pay`	Admin marks a fee record as paid
Create Fee	POST	`/api/fees`	Admin adds a new fee record for a student
Enrollment	POST	`/api/enrollment`	Faculty/Admin enrolls a student in a course
Attendance Request	POST	`/api/attendance-request`	Student requests attendance correction
Get Attendance Requests	GET	`/api/attendance-requests`	Role-filtered list of correction requests
Admin Approve Attendance	PATCH	`/api/attendance-request/:id`	HOD/Admin approves request
Faculty Grant Attendance	PATCH	`/api/attendance-request/:id/mark`	Faculty records the corrected attendance
Resource Booking	POST	`/api/resource-booking`	Faculty requests a resource booking
Get Bookings	GET	`/api/resource-bookings`	Faculty sees own; admin sees all
Approve Booking	PATCH	`/api/resource-booking/:id`	Admin approves/rejects resource booking
Syllabus Progress	GET	`/api/syllabus-progress`	Get syllabus completion per course
Update Syllabus	PATCH	`/api/syllabus-progress`	Faculty updates completion percentage

Architecture Diagram

> **Note:** The NestJS backend will **not** be rewritten. React will continue consuming the same REST endpoints. Only the frontend rendering layer changes.

Task 5: Problems in the Existing Frontend

#	CURRENT ISSUE	WHERE DOES IT OCCUR?	WHY IS IT A PROBLEM?
1	**Massive monolithic JavaScript file (`fixes.js` — 3,059 lines, 208 KB)**	All dashboard pages	The entire frontend logic for all roles (Student, Faculty, HOD, Director, SaaS) is crammed into one file. This makes it difficult to maintain, debug, or extend without risk of breaking unrelated features. Any change requires searching thousands of lines.
2	**Full page reload on every navigation**	`state.js → window.location.href = '...'` on every role redirect and logout	Every role switch, logout, or auth failure triggers a full browser reload. Users experience blank white flashes between pages. There is no concept of client-side routing; the browser discards all state on each navigation.
3	**Entire HTML sections built as template literal strings inside `innerHTML`**	Every render function in `fixes.js` (e.g. `renderStudentCourses`, `renderUsersTable`, `renderTimetableGrid`)	This approach bypasses the browser's incremental DOM update model — it destroys and recreates the entire section on every render call. It also creates XSS vulnerability risks if user data isn't sanitised before injection. There is no diffing; even unchanged parts are thrown away and recreated.
4	**Sidebar, header, and layout HTML duplicated in every page file**	`student.html`, `faculty.html`, `hod.html`, `director.html`	The sidebar HTML (with brand, user avatar, nav items, settings, logout button) is copy-pasted across 4 files. A change to the sidebar structure (e.g. adding a new menu item or updating branding) requires editing 4 separate files. This inevitably leads to inconsistencies.
5	**No client-side state management — all state lives in `localStorage` strings**	`state.js` (SaaSStore, Auth)	Inter-portal communication between `director.html` and `saas.html` is done by reading and writing raw JSON strings into `localStorage`. There is no reactivity — a change in one tab does not update the other. Onboarding leads, support tickets, and activity logs are manually serialised/deserialised with no schema validation.
6	**All form validation duplicated inline in every form handler**	`login.html → handleLogin()`, `fixes.js → submitAddUser()`, `submitEditUser()`	Validation logic (email regex, password strength checks) is copy-pasted in multiple places. A change to validation rules (e.g. adding a new password requirement) must be updated everywhere manually, and inconsistencies will appear.
7	**No component encapsulation — UI and data logic are tightly entangled**	All of `fixes.js`	Every render function fetches data from the API *and* builds the HTML *and* handles events. There is no separation between "data fetching", "presentation", and "event handling". This makes unit testing impossible and code reuse very difficult.

React Conversion Opportunities

FEATURE	CURRENT STATE	REACT IMPROVEMENT
Sidebar	Duplicated HTML in 4 files	Single `<Sidebar role={...} />` component, shared across all pages via React Router layouts

FEATURE	CURRENT STATE	REACT IMPROVEMENT
Data Tables	`innerHTML` template literals	`<DataTable columns={...} rows={...} onEdit={...} onDelete={...} />` component with proper state updates
Modals	`display:flex` / `display:none` toggled by `openModal()`	`<Modal isOpen={state} onClose={fn}>` with conditional rendering
Role Filter Tabs	Inline `<button>` rendering + `currentActiveRoleFilter` global variable	`<FilterTabs tabs={...} active={...} onChange={setFilter} />` with `useState`
API calls + loading states	Manual `el.innerHTML = 'Loading...'` before every fetch	Custom `useFetch('/api/...')` hook returning `{ data, loading, error }`
Page routing	`window.location.href = 'page.html'` (full reloads)	`<Routes>` + `<Route path="/dashboard">` with `<Navigate>` for role-based protection
Toast notifications	`document.createElement('div')` injected into body	`<ToastProvider>` using Context API + `useToast()` hook
Authentication state	`localStorage.getItem('bp_user')` read on every page load	`AuthContext` providing `user`, `login()`, `logout()` to all components

First React Conversion Target

Selected Feature: **Student Dashboard — Courses View**

Reason for selection:

The `renderStudentCourses()` function in `fixes.js` (lines 147–205) is one of the most complex and self-contained render functions in the application. It:

- Makes a real API call (`/api/students/me/courses`)

- Handles loading and error states

- Renders a rich, interactive card layout with nested module progress bars

- Uses inline hover effects via `onmouseover` / `onmouseout`

- Builds deeply nested HTML using template literals

It is an ideal first conversion target because it is fully **isolated** (does not depend on other views), it exercises **data fetching**, **conditional rendering**, and **dynamic styling** — core React concepts.

Initial React Conversion Sketch

```
// CourseCard.jsx — A single reusable course card
function CourseCard({ course }) {
  return (
    <div className="course-card">
      <div className="course-header">
        <span className="badge">{course.course_code}</span>
        <span className="credits">{course.credits} Credits</span>
      </div>
      <h3>{course.course_name}</h3>
      <p className="faculty">{course.faculty_name}</p>
      <div className="metrics">
        <MetricBadge label="Attendance" value={course.attendance_pct} unit="%" />
        <MetricBadge label="Syllabus"   value={course.syllabus_progress} unit="%" />
      </div>
      {course.modules?.map(m => (
        <ModuleProgressBar key={m.name} module={m} />
      ))}
    </div>
  );
}

// CoursesView.jsx — Fetches data and renders a list of cards
function CoursesView() {
  const [courses, setCourses] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);

  useEffect(() => {
    api('/students/me/courses')
      .then(data  => { setCourses(data); setLoading(false); })
      .catch(err  => { setError(err.message); setLoading(false); });
  }, []);

  if (loading) return <LoadingSpinner />;
  if (error)   return <ErrorMessage message={error} />;
  if (!courses.length) return <EmptyState message="No courses enrolled." />;

  return (
    <div className="courses-grid">
      {courses.map(c => <CourseCard key={c.course_id} course={c} />)}
    </div>
  );
}
```

Components Identified for First Conversion:

COMPONENT	PURPOSE
`<CoursesView />`	Fetches and displays the course list with loading/error/empty states
`<CourseCard />`	Renders a single course card with all metrics
`<MetricBadge />`	Reusable metric display (Attendance %, Syllabus %) with colour coding
`<ModuleProgressBar />`	Renders a single module name + progress bar
`<LoadingSpinner />`	Shared loading placeholder (reusable across all views)
`<EmptyState />`	Shared empty data placeholder

Login Credentials

ROLE	EMAIL	PASSWORD	INSTITUTE CODE
Student	`student@iiits.in`	`Pass@123`	`IIITS`
Faculty	`faculty@iiits.in`	`Pass@123`	`IIITS`
HOD / Academic Head	`head@iiits.in`	`Pass@123`	`IIITS`
Institute Director	`director@iiits.in`	`Pass@123`	`IIITS`
SaaS Global Admin	`saasadmin@platform.com`	`Pass@123`	`global`
SaaS Sales Lead	`sales@platform.com`	`Pass@123`	`global`
SaaS Tech Support	`techsupport@platform.com`	`Pass@123`	`global`

Application URLs (Port 5001)

PORTAL	URL
Campus Login	`http://localhost:5001/login.html`
SaaS Support Login	`http://localhost:5001/saas-login.html`
Public Onboarding	`http://localhost:5001/onboarding.html`